

Oyun Yorumlari Uzerinden Duygu Analizi

Tunahan Efe Öztürk

220201038

ozturktunahanefe@gmail.com

Mehmet Ali Demir

220201092

mehmetalidemir1370@gmail.com

Mehmet Yilmaz

210201018

yilmazmehmet7711@gmail.com

Emirhan Aydın

220201099

Aydinnemirhann28@gmail.com

Özet

Bu proje, Steam platformundaki milyonlarca kullanıcı incelemesini büyük veri ve makine öğrenmesi yöntemleriyle analiz eden uçtan uca bir sistem sunmaktadır. Apache Kafka ile canlı veri akışı simüle edilmiş, PySpark ve Delta Lake mimarisiyle veri işleme ve depolama süreçleri optimize edilmiştir. Projenin en kritik aşamasında, veri setindeki %82'lik olumlu yorum yoğunluğunun yarattığı sınıf dengesizliği problemi Alt Örneklem (Undersampling) yöntemiyle çözülmüştür. Yapılan kapsamlı model kıyaslamaları sonucunda, Linear SVC (Doğrusal Destek Vektör Sınıflandırması) algoritması; %74,09 Doğruluk (Accuracy) ve %75,31 F1-Skoru ile en başarılı model olarak belirlenmiştir. Seçilen bu model ile oyun bazında duygu analizi gerçekleştirilmiş ve sonuçlar interaktif görselleştirme araçlarıyla doğrulanmıştır.

Giriş

Steam gibi devasa platformlarda her gün binlerce yeni kullanıcı incelemesi yayınlanmaktadır. Bu verilerin manuel olarak incelenmesi imkansız olduğundan, otomatik duygu analizi sistemleri oyun geliştiricileri ve potansiyel alıcılar için kritik önem taşır. Bu çalışma, ham metin verisinin kaynağından alınıp işlenmesinden, karmaşık makine öğrenmesi modelleriyle tahmin üretilmesine kadar olan tüm süreçleri endüstriyel standartlarda bir veri hattı (data pipeline) ile gerçekleştirmeyi amaçlar. Çalışmanın odak noktası, ham verideki gürültüyü temizleyerek ve sınıfları dengeleyerek en yüksek tahmin gücüne sahip algoritmayı tespit etmek ve bunu büyük veri üzerinde uygulamaktır.

Yöntem

Bu çalışma, ham verinin kaynağından alınmasından model sunumuna kadar uzanan modern bir büyük veri analitiği boru hattı (data pipeline) üzerine inşa edilmiştir. Yöntem; veri akışı, veri gölü (data lake) katmanlandırması, özellik mühendisliği, sınıf dengeleme ve makine öğrenmesi modellemesi olmak üzere beş temel aşamada yürütülmüştür:

3.1. Gerçek Zamanlı Veri Akışı (Real-Time Data Streaming)

Projenin veri kaynağı olan milyonlarca satırlık Steam kullanıcı incelemeleri, statik bir dosya okuma işlemi yerine uçtan uca bir veri akışı senaryosuyla sisteme entegre edilmiştir. Bu amaçla Docker konteynerleri üzerinde Apache Kafka mimarisi ayağa kaldırılmış ve bir producer.py betiği aracılığıyla CSV dosyasındaki veriler mikro-partiler (micro-batches) halinde Kafka topic'lerine gönderilerek canlı bir veri akışı simüle edilmiştir. Bu akış, PySpark Structured Streaming teknolojisi ile eşzamanlı olarak dinlenmiş ve tüketilmiş, böylece büyük verinin gerçek zamanlı işleme kapasitesi test edilmiştir.

3.2. Delta Lake Mimarisi ile Katmanlı Veri Depolama (Data Storage & Preprocessing)

Databricks ortamına alınan veriler, veri bütünlüğünü ve ACID işlemlerini garanti eden Delta Lake formatında üç farklı mimari katmanda işlenmiştir:

- **Bronze Katmanı (Ham Veri):** Kafka'dan gelen JSON formatındaki veri paketleri çözümlenerek hiçbir filtreleme yapılmadan doğrudan bu katmana kaydedilmiştir.
- **Silver Katmanı (Temiz Veri):** Ham veri üzerinde tip dönüşümleri (casting) yapılmış, analizde kullanılmayacak gereksiz sütunlar düşürülmüş (drop) ve içinde "NaN" (Not a Number) veya boş veri (Hidden Nulls) barındıran satırlar temizlenerek veri kalitesi artırılmıştır.
- **Features Katmanı (Makine Öğrenmesine Hazır Veri):** Özellik mühendisliği adımlarının uygulandığı ve model eğitime doğrudan verilebilecek son formattaki tablodur.

3.3. Özellik Mühendisliği (Feature Engineering)

Modellerin metin verilerini (review) matematiksel olarak anlamlandırabilmesi için PySpark fonksiyonları kullanılarak yeni öznitelikler üretilmiştir. Her bir yorum için;

- Yorumun toplam karakter uzunluğu (review_length),
- Yorumdaki toplam kelime sayısı (word_count),
- Yorumun diğer kullanıcılar tarafından faydalı bulunup bulunmadığı (is_voted),
- Regex ifadeleri yardımıyla yorumun içinde geçen olumlu ("good, great, love, vb.") ve olumsuz ("bad, terrible, bug, vb.") anahtar kelimelerin varlığı ikili (binary) değişkenler (has_positive_word, has_negative_word) olarak veri setine kazandırılmıştır. Çıkarılan bu özellikler, VectorAssembler ile tek boyutlu bir özellik vektörüne dönüştürülmüştür.

3.4. Sınıf Dengesizliği (Class Imbalance) ve Alt Örnekleme (Undersampling)

Keşifçi Veri Analizi (EDA) aşamasında, Steam veri setindeki yorumların yaklaşık %82'sinin "Olumlu (1)", %18'inin ise "Olumsuz (0)" olduğu tespit edilmiştir. Bu dengesizlik, modellerin sürekli "Olumlu" tahmini yaparak yapay bir yüksek Doğruluk (Accuracy Paradox) elde etmesine ve ezber yapmasına sebep olmuştur. Bu problemi çözmek için veri seti öncelikle %80 Eğitim (Train) ve %20 Test olarak ikiye ayrılmıştır. Modelin gerçek dünya performansını doğru ölçmek adına Test setine dokunulmamış; ancak Eğitim seti üzerinde "Alt Örnekleme (Undersampling)" tekniği uygulanmıştır. Çoğunluk sınıfı olan olumlu yorumlar rastgele seyreltilerek olumsuz yorum sayısı ile eşitlenmiş ve modellere %50-%50 oranında adil ve dengeli bir öğrenme ortamı sunulmuştur.

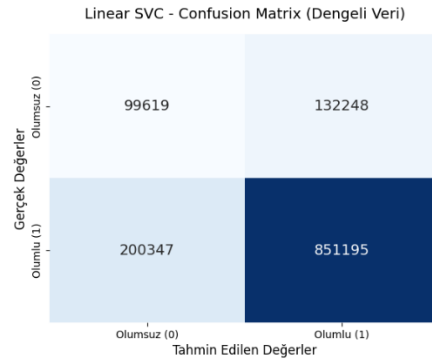
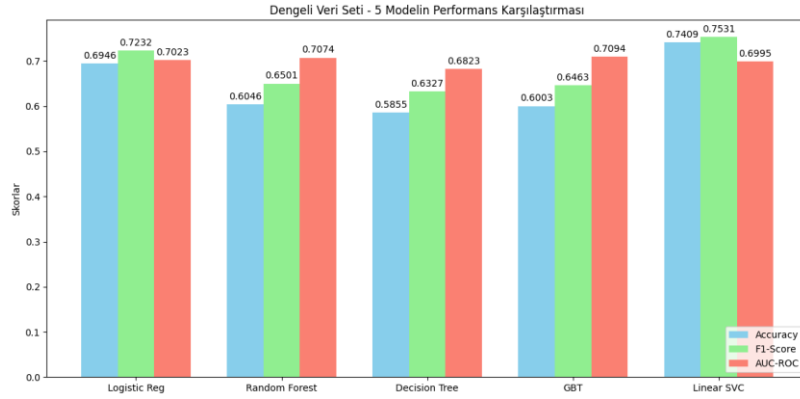
3.5. Model Eğitimi, İzleme (MLflow) ve Değerlendirme

Dengeli eğitim seti kullanılarak PySpark MLlib kütüphanesinden beş farklı sınıflandırma algoritması eğitilmiştir: *Logistic Regression*, *Random Forest*, *Decision Tree*, *Gradient Boosted Trees (GBT)* ve *Linear SVC*. Model eğitimi sırasında hiperparametreler (maksimum derinlik, ağaç sayısı, iterasyon) ayarlanmış ve her bir modelin performansı MLflow ile takip edilerek "Accuracy, F1-Score ve AUC-ROC" metrikleri otomatik olarak loglanmıştır.

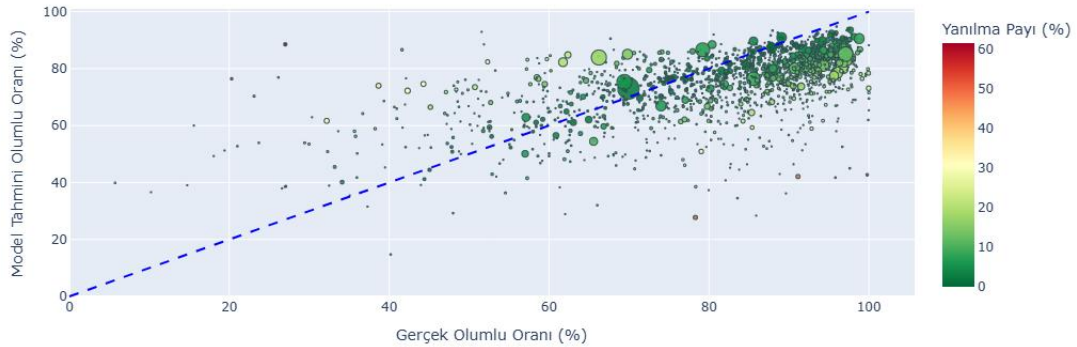
Performans değerlendirmeleri sonucunda, Linear SVC (Doğrusal Destek Vektör Sınıflandırması) algoritması en yüksek başarımlarına ulaşmıştır. Linear SVC algoritması doğası gereği standart bir olasılık (probability) yüzdesi üretmese de, modelin karar sınırına (hiperdüzleme) olan uzaklığını ifade eden rawPrediction skorları kullanılarak hata analizi, Karmaşıklık Matrisi (Confusion Matrix) ve ROC Eğrileri başarıyla hesaplanmış; son aşamadaki veri görselleştirme paneli (dashboard) bu skorlar baz alınarak oluşturulmuştur.

Sonuç

Proje sonucunda elde edilen veriler, sınıf dengesizliği giderildiğinde modellerin tahmin tutarlılığının arttığını kanıtlamıştır. Linear SVC modeli, özellikle F1-Skoru ve genel doğruluk bakımından diğer tüm rakiplerini geride bırakarak projenin en başarılı algoritması olmuştur. Modelin oyun bazlı tahminleri, Steam'deki gerçek kullanıcı puanlarıyla karşılaştırıldığında yüksek korelasyon göstermiş ve "Kusursuz Tahmin ($y=x$)" çizgisine yakın sonuçlar üretmiştir. Linear SVC'nin bu başarısı, özellik mühendisliğiyle çıkarılan sayısal değişkenlerin doğrusal bir düzlemde sınıflandırmaya oldukça yatkın olduğunu ispatlamıştır. Bu çalışma, büyük veri ekosisteminde doğru veri mühendisliği adımlarının, en karmaşık algoritmalar yerine veriye en uygun modeli (Linear SVC) başarıyla seçmemize nasıl olanak tanıdığını göstermektedir.



Linear SVC: Gerçek vs. Tahmini Beğeni Oranı Karşılaştırması



Kaynakça

<https://www.kaggle.com/datasets/andrewmvd/steam-reviews>
<https://docs.databricks.com/aws/en/getting-started/>
<https://kafka.apache.org/42/getting-started/introduction/>